Проектирование API для ML-сервисов (FastAPI)

Арзамасова Анна
Эксперт курса «Интеграция ИИ»



Проверка связи

Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти

Поставьте в чат:

\+ если меня видно и слышно

— если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры Google Chrome или Microsoft Edge
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение на телефоне
- Предварительно проверьте, подходит ли ваше устройство для подключения к вебинару, по ссылке

Анна Арзамасова

О спикере:

- инженер в сфере авиации, сотрудник НИИ
- специалист по интеллектуальным системам управления транспортом
- преподаватель Московского Политеха
- Преподаватель и разработчик образовательных программ на стыке инженерии, программирования и искусственного интеллекта



Начинаем модуль 3 · Разбор ДЗ модуля 2

ОРГМОМЕНТ

От тестов — к реальному сервису

Модуль 2 научил проверять данные, модель и код вокруг неё.

Модуль 3 — как превратить это в реально работающий сервис.

Цели занятия

- **Научимся** давать модели голос:
  превращать предсказание в сервис, доступный другим системам.
- **Разберём** путь запроса:
  клиент → API → валидация → модель → ответ.
- **Сможем** создавать понятный ML-сервис с контрактом, проверкой входных данных и документацией.

План вебинара

1. REST vs gRPC vs GraphQL
2. Специфика ML-сервисов
3. FastAPI: pydantic и dependency injection
4. Практика: живой код — ML-сервис на FastAPI
5. Итоги

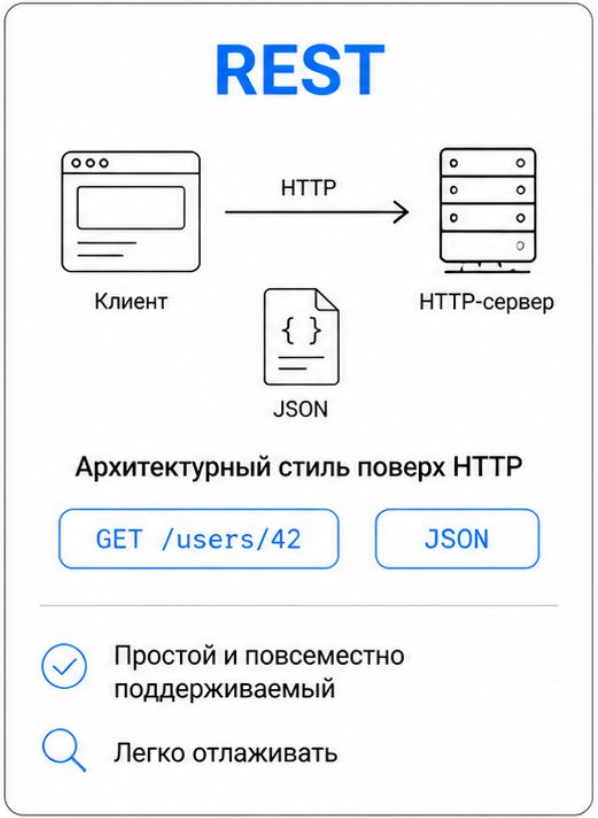
REST, gRPC, GraphQL

ЧАСТЬ 1

REST

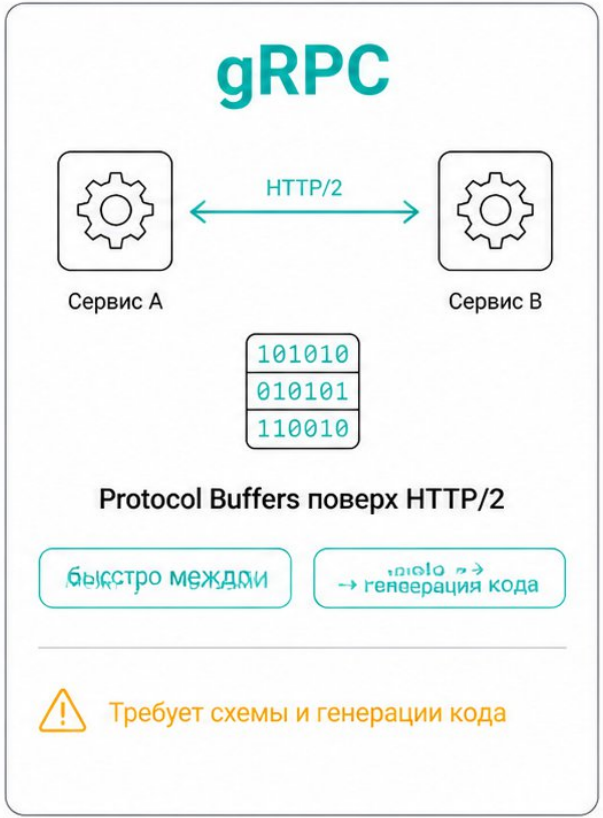
Архитектурный стиль поверх HTTP:
запрос на эндпоинт с методом,
ответ в JSON.

Простой, повсеместно поддерживаемый,
легко отлаживаемый.



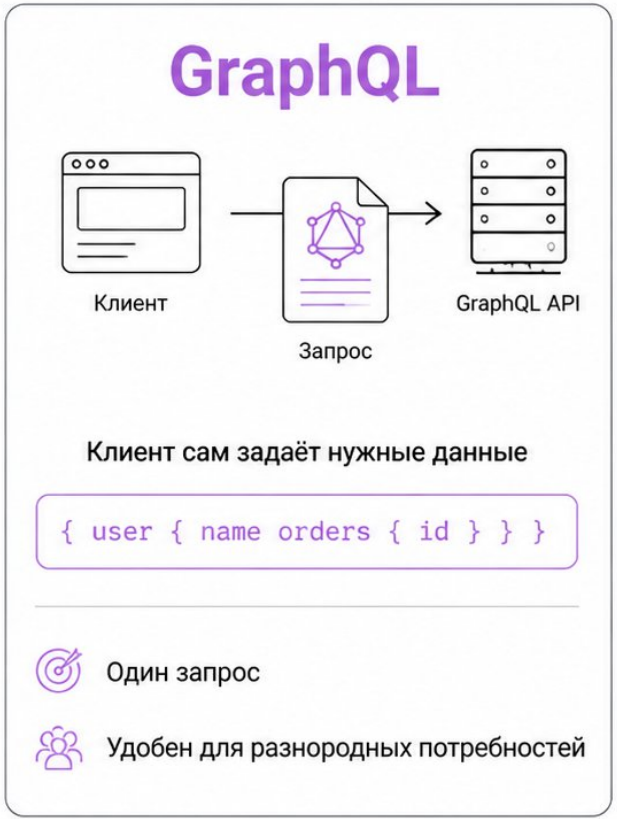
gRPC

Бинарный протокол (Protocol Buffers) поверх HTTP/2 для быстрого взаимодействия между сервисами внутри инфраструктуры.
Быстрее REST, но требует генерации кода из схемы.



GraphQL

Клиент сам описывает, какие данные ему нужны, в одном запросе.
Удобен при сложных разнородных потребностях в данных.



Для большинства ML-сервисов REST — разумный выбор по умолчанию: прост в реализации, отладке и интеграции

Пример



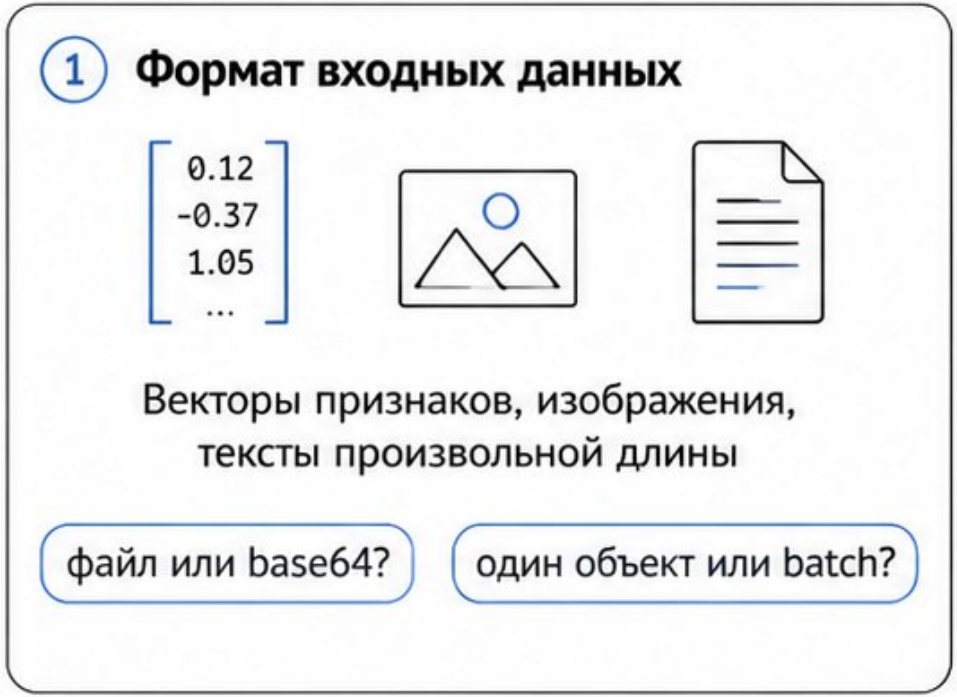
Один OCR-сервис — разные интерфейсы для разных сценариев

Специфика ML-сервисов

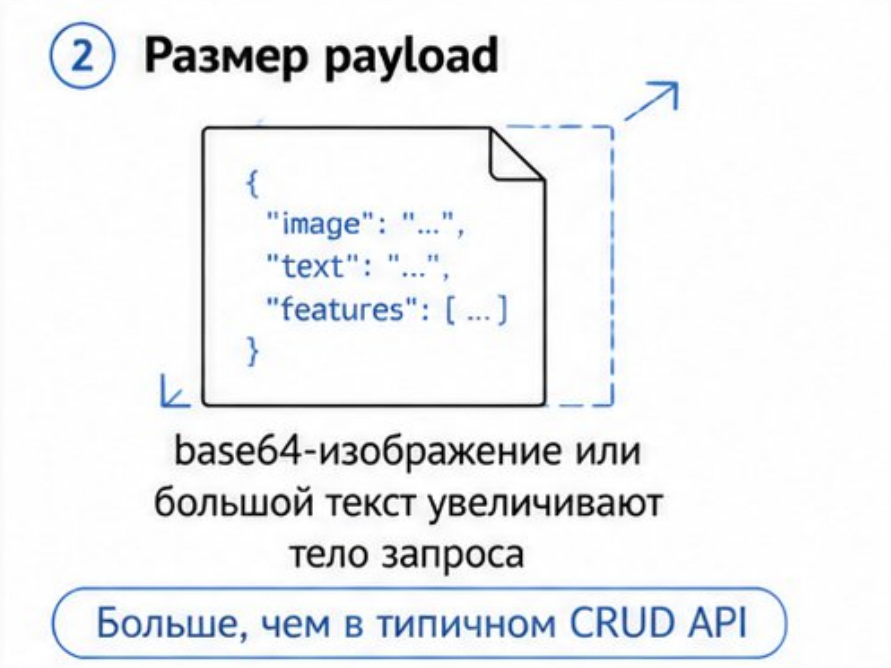
ЧАСТЬ 2

ML-сервис — тоже REST API, но со своей спецификой по сравнению с обычным CRUD-сервисом

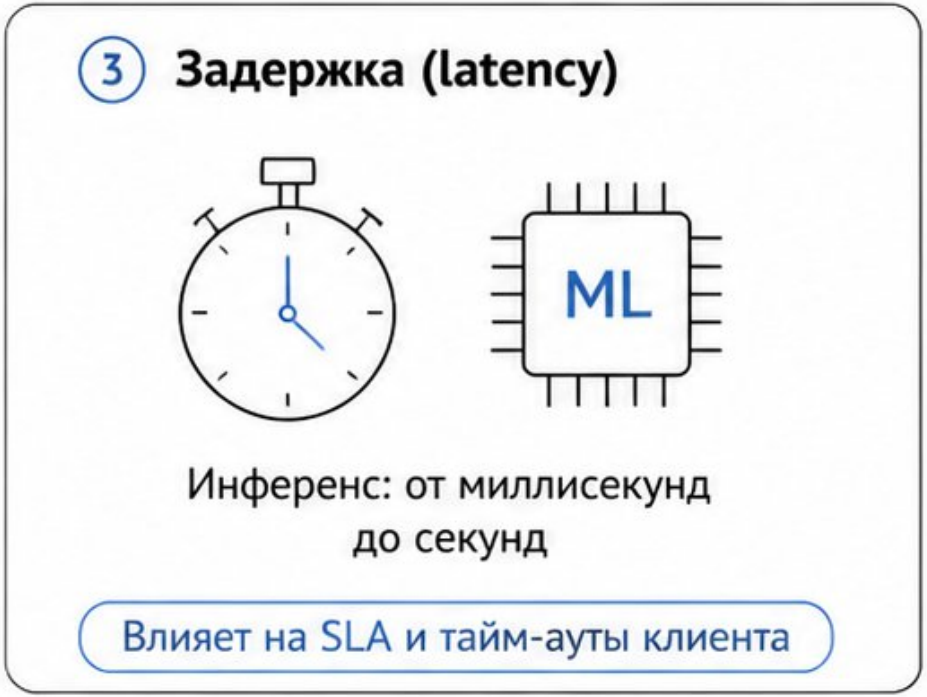
Формат входных данных



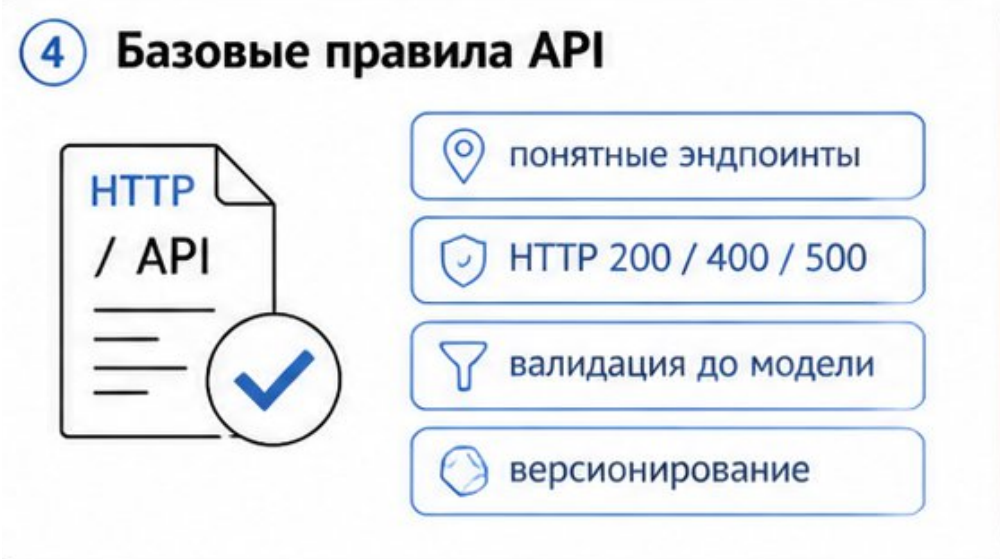
Размер payload



Задержка (latency)



Что не меняется



Пример

5 /predict: плохой и хороший контракт

Плохо

`Произвольный JSON → 500`



Сервис падает без понятной причины

Хорошо

`Точная схема Pydantic → 422`



Указано проблемное поле

```
amount: float > 0
```

FastAPI: почему и основные концепции

ЧАСТЬ 3

Почему FastAPI для ML-сервисов

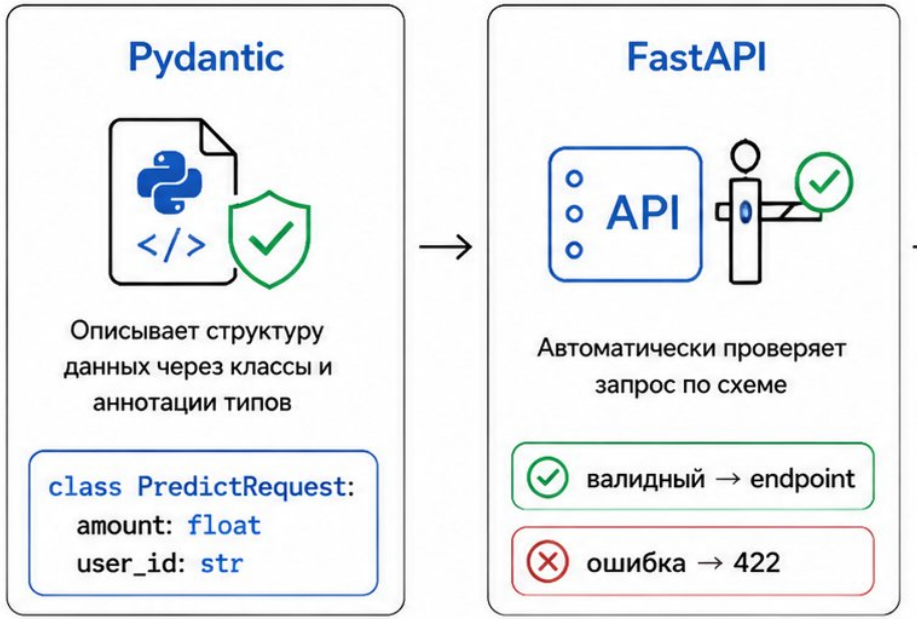
Автоматическая валидация входных данных по схеме

Автоматическая документация (Swagger UI) без доп. кода

Высокая производительность за счёт асинхронной обработки

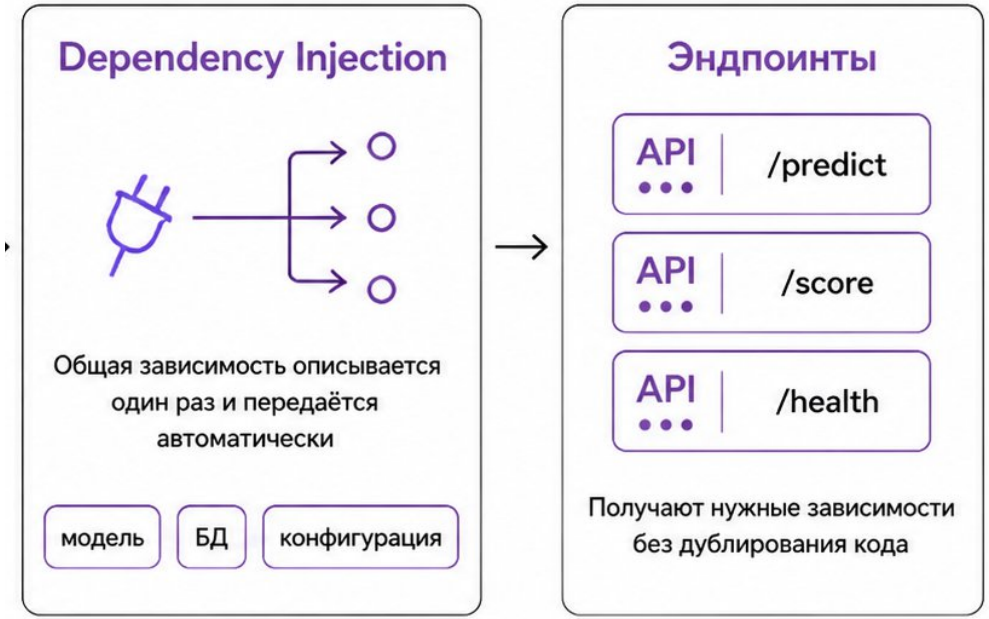
Pydantic

Библиотека для описания структуры данных через классы с аннотациями типов. FastAPI автоматически проверяет запрос по этой структуре.

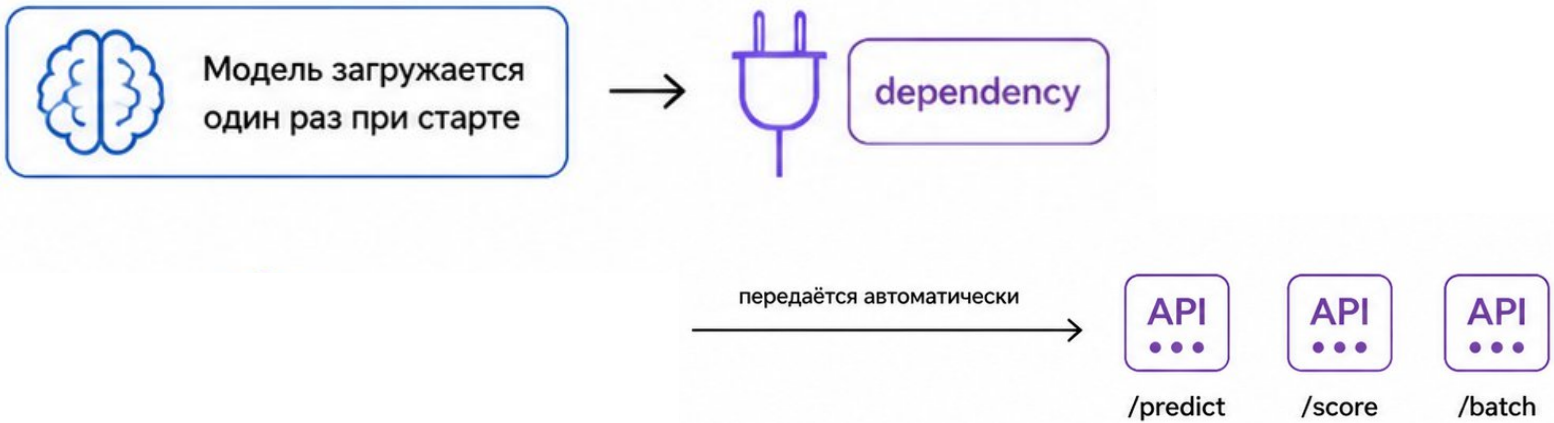


Dependency injection

Общие для нескольких эндпоинтов вещи описываются как «зависимость» один раз и передаются автоматически, без дублирования кода.



Пример



Pydantic защищает вход, Dependency Injection переиспользует общие ресурсы

Pydantic описывает схему декларативно, dependency injection убирает дублирование кода загрузки общих ресурсов

Живой код: ML-сервис на FastAPI

ПРАКТИКА

Шаг 1

Устанавливаем FastAPI и uvicorn — сервер, который запускает FastAPI-приложение.

```
1 !pip install fastapi uvicorn httpx nest_asyncio scikit-learn --quiet
2 print('Библиотеки установлены')
```

```
Библиотеки установлены
```

Шаг 2

Загружаем готовую обученную модель (pickle/joblib).

```python
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pickle

X, y = make_classification(n_samples=1000, n_features=6, n_informative=4, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
acc = accuracy_score(y_test, model.predict(X_test))
print("Accuracy на тесте:", round(acc, 3))

MODEL_VERSION = "1.0.0"
with open("model.pkl", "wb") as f:
    pickle.dump({"model": model, "version": MODEL_VERSION, "accuracy": acc}, f)

print("Модель сохранена в model.pkl")
```

```
Accuracy на тесте: 0.925
Модель сохранена в model.pkl
```

Шаг 3

Описываем pydantic-модели для запроса и ответа /predict.

```python
from pydantic import BaseModel, Field
from typing import List

class PredictRequest(BaseModel):
    features: List[float] = Field(..., min_length=6, max_length=6, description="Ровно 6 числовых признаков")

class PredictResponse(BaseModel):
    probability: float
    decision: str

class ModelInfo(BaseModel):
    version: str
    accuracy: float
    n_features: int
```

Три эндпоинта сервиса

/predict — инференс модели
JSON на входе, предсказание на выходе

/healthcheck — проверка, что сервис жив и модель загружена

/model-info — метаданные модели
версия, дата обучения, метрика

Шаг 5

Запускаем сервис через uvicorn (nest_asyncio для Colab), проверяем через Swagger UI и TestClient.

```
[ ]
import nest_asyncio
import uvicorn
import threading

nest_asyncio.apply()

def run_server():
    uvicorn.run(app, host="127.0.0.1", port=8000, log_level="warning")

server_thread = threading.Thread(target=run_server, daemon=True)
server_thread.start()

import time
time.sleep(2)

import httpx
resp = httpx.get("http://127.0.0.1:8000/healthcheck")
print("Сервер запущен, ответ:", resp.status_code, resp.json())
print("Документация доступна по адресу http://127.0.0.1:8000/docs (внутри среды Colab)")
```

```
Сервер запущен, ответ: 200 {'status': 'ok', 'model_loaded': True}
Документация доступна по адресу http://127.0.0.1:8000/docs (внутри среды Colab)
```

Шаг 6. Самостоятельно

Добавьте валидацию входных данных через pydantic: например, ограничение диапазона через Field(ge=0, le=1).

Если не работает

uvicorn не запускается
nest_asyncio.apply() перед запуском сервера

Swagger UI не открывается
используйте TestClient — не требует реального сетевого сервера

Ошибка валидации на «корректном» запросе
проверьте типы — число как строка вместо числа

Вы узнали

- → API соединяет модель с пользователем и остальной системой
- → REST подходит большинству ML-сервисов: он прост, прозрачен и удобен для интеграции
- → Валидация защищает модель от некорректных входных данных
- → FastAPI помогает быстро собрать сервис, документацию и проверяемый контракт

Главная мысль:

Модель становится частью продукта в тот момент, когда её результат может получить другой человек или сервис.

Что мы разобрали сегодня

ИТОГИ

REST — разумный выбор по умолчанию для большинства ML-сервисов; gRPC и GraphQL — для специфичных сценариев

ML-сервис имеет особенности: формат входа, размер payload, задержка инференса — закладываются в проектирование заранее

FastAPI даёт валидацию через pydantic, документацию через Swagger UI и производительность через асинхронность

Домашнее задание

Реализуйте FastAPI-сервис для модели вашего проекта — минимум 3 эндпоинта, проверьте через Swagger UI.

Мост к вебинару 10

На следующем вебинаре — как выкатить обновлённую версию сервиса, не сломав текущих пользователей: blue-green, canary, A/B-тестирование, откат.

СПАСИБО

До встречи на вебинаре 10

Проектирование API для ML-сервисов (FastAPI)

Арзамасова Анна
Эксперт курса «Интеграция ИИ»

